`maxentcpp`: A C++ reimplementation of Maximum Entropy species distribution modeling for R

Ángel Luis Robles Fernández*

Department of Ecology and Evolutionary Biology,
University of Kansas, Lawrence, KS, United States

May 2026

1 Summary

`maxentcpp` is an R package that provides a high-performance C++17 reimplementation of the Maximum Entropy (Maxent) algorithm for species distribution modeling (SDM). Maxent estimates the geographic distribution of a species from presence-only occurrence records and environmental covariates by finding the probability distribution of maximum entropy subject to constraints derived from the training data [Phillips et al., 2006, 2004]. Since its introduction, Maxent has become one of the most widely cited methods in ecology and conservation biology, with over 13 000 citations [Elith et al., 2011].

The fitted model is a *Gibbs distribution* over geographic space:

$$P(\mathbf{x}) \;=\; \frac{1}{Z}\,\exp\Big(\sum_{j=1}^{J}\lambda_j\, f_j(\mathbf{x})\Big), \tag{1}$$

where f_j are feature transformations of the environmental variables, λ_j are learned weights, and $Z = \sum_{\mathbf{x}} \exp(\sum_j \lambda_j f_j(\mathbf{x}))$ is the normalizing constant (partition function). The optimizer finds the λ_j that maximize the regularized log-likelihood via sequential coordinate ascent with ℓ_1 (lasso) penalties [Phillips et al., 2006, Elith et al., 2011].

The original Maxent software is a Java desktop application [Phillips et al., 2017]. `maxentcpp` translates the complete Maxent 3.4.4 algorithm into C++17 with R bindings via Rcpp [Eddelbuettel and François, 2011] and RcppEigen [Bates and Eddelbuettel, 2013], eliminating the Java Runtime Environment dependency entirely. The package supports all five feature types (linear, quadratic, product, hinge, and threshold), output transformations in raw, logistic, and complementary log-log (cloglog) scales, and a streaming raster evaluation engine that processes environmental layers block-by-block through the `terra` package [Hijmans, 2024], enabling continental-scale projections on commodity hardware without loading entire raster stacks into memory. Model diagnostics include AUC evaluation, permutation importance, response curves, and Multivariate Environmental Similarity Surfaces (MESS) for novelty detection.

2 Statement of need

Maxent is the *de facto* standard for presence-only species distribution modeling [Merow et al., 2013]. However, the original Java implementation presents several practical barriers for modern ecological research workflows:

*Corresponding author. ORCID: 0000-0002-4741-8012. E-mail: `a.l.robles.fernandez@gmail.com`

1. **Java dependency.** Users must install and configure a compatible JDK. Version conflicts between Java 8, 11, 17, and 21 produce silent failures, and the `rJava` bridge package is one of the most common sources of installation errors in the R ecosystem, particularly on macOS and HPC clusters. While `conda` environments and Docker containers can manage Java versioning, they add deployment complexity and are not standard practice in many ecology research groups.
2. **File-based I/O.** Data must be serialized to disk as SWD or ASCII grid files before the Java process can read them, and results must be parsed back from output files. There is no in-memory bridge between R data structures and the Java optimizer.
3. **Scalability.** The Java implementation is single-threaded and GUI-centric. In multi-species, multi-scenario workflows (e.g., 500 species $\times$ 4 climate scenarios), file I/O overhead accumulates: each species run requires writing SWD files, invoking the JVM, and parsing output files. A native in-memory API eliminates this per-call overhead entirely.
4. **Maintenance.** The Java Maxent codebase has remained essentially unchanged since version 3.4.4 (released November 2020), with only two active contributors [Phillips et al., 2017].

`maxentcpp` addresses these barriers by providing a native C++/R implementation that installs with a single `install.packages("maxentcpp")` call, operates entirely in memory through R objects, and integrates seamlessly with the `terra` spatial data ecosystem. The target audience is ecologists, conservation biologists, and biogeographers who use Maxent in reproducible, script-based workflows—from individual species analyses to large-scale biodiversity assessments.

3 Legacy software modernization in ecology

Computational ecology relies on a small number of foundational software tools, many of which were written over a decade ago and have not been modernized for current computing environments. Reimplementing legacy scientific software in modern languages is uncommon in ecology but has important precedents in adjacent environmental sciences.

Circuitscape, the standard tool for landscape connectivity modeling, was rewritten from Python to Julia, achieving order-of-magnitude speedups while preserving result equivalence [Hall et al., 2021]. The LANDIS-II forest landscape model was re-engineered from monolithic C into a modular C# framework with automated testing and version control [Scheller et al., 2009]. Most recently, the WaterGAP global hydrological model was reprogrammed from Fortran to Python with systematic cross-validation against the original [Nyenah et al., 2025]. Closer to species distribution modeling, `maxnet` [Phillips, 2024] represents the original MaxEnt author's own reimplementation using R and `glmnet`—preserving the same statistical model but employing a different optimization algorithm (coordinate descent on the full feature matrix rather than sequential coordinate ascent).

These precedents share common motivations: eliminating obsolete runtime dependencies, enabling integration into modern scripted workflows, improving maintainability, and opening the algorithm to inspection and extension. `maxentcpp` follows this lineage but goes further by **preserving the original optimization algorithm** (sequential coordinate ascent with ℓ_1-penalized Newton steps) rather than substituting an approximate equivalent, and by **validating per-iteration numerical equivalence** rather than only comparing final-state outputs.

To our knowledge, `maxentcpp` is the first project in ecology to systematically reimplement a widely used SDM algorithm in a compiled language with per-iteration numerical validation against the original implementation, combining faithful algorithm porting with quantitative fidelity testing as a companion package (`maxentcppCompTest`).

4 State of the field

Several R packages provide access to Maxent-family models. **dismo** [Hijmans et al., 2023] is the most widely used R interface to Java Maxent; it wraps `maxent.jar` via `rJava`, inheriting the JDK dependency and file I/O overhead, and is currently in maintenance mode following the transition from `raster` to `terra`. **maxnet** [Phillips, 2024] is a pure-R implementation that uses `glmnet` for elastic-net regularization; while it avoids the Java dependency, it employs a different optimization algorithm (elastic net via coordinate descent on the full feature matrix) rather than the sequential coordinate-ascent optimizer of the original Maxent, which can produce numerically different results, particularly for threshold and hinge features. **ENMeval** [Kass et al., 2021] is a model tuning and evaluation framework that wraps either `dismo::maxent()` or `maxnet::maxnet()`; it does not implement the Maxent algorithm itself. **kuenm** [Cobos et al., 2019] is a calibration and evaluation toolkit that relies on `dismo` or direct Java Maxent calls.

4.1 Empirical comparison: maxentcpp vs maxnet

To quantify the practical differences between `maxentcpp` and `maxnet`, we compared both packages on a virtual species [Leroy et al., 2016] with known true habitat suitability, constructed from the environmental layers bundled with `maxentcpp` (Annual Mean Temperature and Annual Precipitation over Central America, 2,371 non-NA cells). The virtual species has a Gaussian niche centered at 20 °C and 1,500 mm, and 100 presence records were sampled proportionally to the true suitability surface (see the companion vignette *Virtual Species Comparison*[1] for full reproducible code).

Both packages were fitted with linear, quadratic, and hinge features. The results demonstrate high ecological equivalence:

Table 1: Ecological equivalence metrics: `maxentcpp` vs `maxnet` on a virtual species.

Metric	Value
Schoener's D	0.93
Spearman ρ (rank correlation)	0.95
Pearson r	0.97
Mean $\lvert\Delta\text{cloglog}\rvert$	0.053
Max $\lvert\Delta\text{cloglog}\rvert$	0.43

These results confirm that `maxentcpp` and `maxnet` produce **ecologically equivalent predictions** for typical modeling scenarios (Schoener's $D > 0.9$ is conventionally interpreted as high niche overlap). The largest differences occur in the tails of the suitability distribution, where the two optimization algorithms (`maxentcpp`: sequential coordinate ascent; `maxnet`: elastic-net coordinate descent) regularize differently. Both packages recover the true niche with comparable accuracy (Spearman $\rho > 0.87$ vs true surface).

The key practical scenarios where a user must choose `maxentcpp` over `maxnet` are: (1) when exact reproduction of Java Maxent results is required (e.g., replicating published analyses), (2) when lambda file interoperability with Java Maxent is needed, and (3) when streaming raster projection onto grids larger than available RAM is required.

4.2 Performance benchmarks

Training time for the bundled *Abeillia abeillei* dataset (73 presences, 2,371 background, linear + quadratic + hinge features):

[1] https://alrobles.github.io/maxentcpp/articles/virtual_species_comparison.html

Table 2: Training time comparison.

Package	Median time per fit
`maxentcpp`	∼820 ms
`maxnet`	∼530 ms

`maxnet` is faster for small datasets because `glmnet`'s coordinate descent is highly optimized for dense feature matrices. However, `maxentcpp`'s streaming evaluation avoids materializing the full prediction matrix, giving it an advantage for projection onto large rasters where `dismo`/Java Maxent would require disk-based intermediates.

`maxentcpp` was built as a new package rather than contributing to existing projects for three reasons. First, a faithful port of the original Java optimizer (sequential coordinate ascent with ℓ_1-penalized Newton steps) requires a C++ engine that cannot be expressed through `glmnet`'s API; contributing this to `maxnet` would change its core algorithm. Second, `dismo`'s architecture is tightly coupled to `rJava` and `raster`; the native C++ approach eliminates this dependency chain entirely. Third, the header-based C++ library design of `maxentcpp` enables reuse in other packages or standalone applications beyond R—something not achievable through modifications to existing R-only packages.

5 Software design

5.1 Architecture

`maxentcpp` is organized in three layers (C++ core, Rcpp bridge, R interface), with the C++ core further split into algorithmic and infrastructure sublayers:

Table 3: Architecture of `maxentcpp`: three layers with the C++ core divided into algorithmic and infrastructure sublayers.

Layer	Key components	Lines
C++ core (algorithmic)	`Sequential`, `FeaturedSpace`, five feature types	∼4 500
C++ core (I/O, diagnostics)	`BackgroundProvider`, grid I/O, CSV, MESS, response curves	∼2 400
Rcpp bridge	`rcpp_*.cpp` external-pointer bindings [Eddelbuettel, 2013]	∼3 300
R interface	`maxent_run()`, feature generators, projection, evaluation, diagnostics	∼2 500

The C++ core uses Eigen [Guennebaud et al., 2010] for dense linear algebra. Of the ∼6 900 C++ lines, approximately 4 500 implement the core algorithmic logic (optimizer, features, density) while the remainder handles I/O, Rcpp bindings, and diagnostics. The high-level `maxent_run()` function provides a one-call workflow mirroring the Java GUI experience, while lower-level functions (`maxent_generate_features()`, `maxent_featured_space()`, `maxent_fit()`, `maxent_project_cloglog()`) offer full control over each modeling step.

5.2 Optimization algorithm

The `Sequential` optimizer is a direct port of `density.Sequential` in Java Maxent 3.4.4. It minimizes the ℓ_1-regularized negative log-likelihood:

$$\mathcal{L}(\boldsymbol{\lambda}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{j=1}^{J}\lambda_j\, f_j(\mathbf{x}_i) + \log Z(\boldsymbol{\lambda}) + \sum_{j=1}^{J}\beta_j\,|\lambda_j|, \tag{2}$$

where m is the number of presence records, $Z(\boldsymbol{\lambda}) = \sum_{k=1}^{N}\exp\!\left(\sum_j \lambda_j f_j(\mathbf{x}_k)\right)$ is the partition function summed over N background points, and $\beta_j = \hat{\sigma}_j/\sqrt{m}$ is the per-feature regularization

parameter with $\hat{\sigma}_j$ being the standard deviation of feature j over the presence points (floored at 0.001).

At each iteration, the optimizer selects the feature j^* with the most negative $\Delta\mathcal{L}$ bound (the `deltaLossBound` function), then computes a Newton step:

$$\alpha_j = -\frac{\partial\mathcal{L}/\partial\lambda_j}{\mathbf{u}_j^\top \mathbf{H}\,\mathbf{u}_j}, \tag{3}$$

where $\mathbf{u}_j$ is the j-th coordinate direction and $\mathbf{u}_j^\top \mathbf{H}\,\mathbf{u}_j = \mathrm{Var}_q[f_j]$ is the variance of feature j under the current distribution q. The derivative includes the ℓ_1 subgradient: $\partial\mathcal{L}/\partial\lambda_j = \mathbb{E}_q[f_j] - \mathbb{E}_{\hat{p}}[f_j] + \beta_j\,\mathrm{sign}(\lambda_j)$. The step is damped during early iterations ($\alpha \leftarrow \alpha/50$ for iterations < 10, $\alpha/10$ for < 20, $\alpha/3$ for < 50) and falls back to a line search (`searchAlpha`) if the Newton step does not decrease loss. Every 10 iterations, a parallel update applies coordinate steps to all features simultaneously.

Convergence is tested every 20 iterations: training stops when the relative loss improvement falls below 10^{-5} or 500 iterations are reached.

5.3 Streaming raster evaluation

`maxentcpp` reads `terra::SpatRaster` objects block-by-block through a `BackgroundProvider` abstraction. Each tile is scored by the C++ engine and discarded before the next tile is loaded, allowing projection onto raster stacks larger than available RAM. The partition function Z is accumulated across tiles by summing unnormalized densities $\exp(\sum_j \lambda_j f_j(\mathbf{x}_k))$ for each cell k in the tile, then normalizing once after all tiles are processed. This produces identical results to single-pass evaluation because the sum is commutative and no tile-boundary effects exist—each cell is scored independently.

5.4 Numerical fidelity

Every C++ class is a direct port of the corresponding Java class, preserving the same control flow, regularization logic, and numerical constants (Table 4). This design choice prioritizes backward compatibility: users migrating from Java Maxent obtain identical results. The fidelity contract is enforced by a dedicated companion package, `maxentcppCompTest` [Fernández, 2025], which runs the C++ optimizer and the original Java `density.Sequential` on shared test fixtures and compares per-iteration trajectories of loss, entropy, and λ vectors.

On controlled test fixtures (identical input data, same floating-point representation), agreement reaches $< 10^{-14}$ relative error for symmetric fixtures and $< 10^{-6}$ on $\|\Delta\lambda\|_\infty$ for asymmetric fixtures at every iteration checkpoint. **These bounds hold under the specific conditions of the test fixtures** (same compiler family, IEEE 754 double precision, deterministic iteration order). Floating-point ordering differences introduced by alternative BLAS backends or aggressive compiler optimizations (e.g., `-ffast-math`) could widen the gap, though the sequential nature of the coordinate-ascent algorithm limits sensitivity to parallelism-induced reordering. Cross-platform CI (Ubuntu, macOS, Windows with GCC, Clang, and MSVC) confirms that the test fixtures pass on all three compiler families.

Lambda file compatibility. Trained models are serialized in the same `.lambdas` text format used by Java Maxent 3.4.4. Lambda files produced by either implementation can be loaded by the other, ensuring interoperability with existing workflows and archived models.

5.5 Feature completeness

The following table summarizes the current scope of `maxentcpp` relative to Java Maxent 3.4.4 and `maxnet`:

Table 4: Class-level porting map from Java Maxent to `maxentcpp`.

`maxentcpp` (C++)	Java Maxent original
`LinearFeature`	`density.LinearFeature`
`QuadraticFeature`	`density.SquareFeature`
`ProductFeature`	`density.ProductFeature`
`HingeFeature`	`density.HingeFeature`
`ThresholdFeature`	`density.ThresholdFeature`
`FeaturedSpace`	`density.FeaturedSpace`
`Sequential::run()`	`density.Sequential.run()`

Table 5: Feature completeness: `maxentcpp` vs Java Maxent vs `maxnet`.

Feature	`maxentcpp`	Java Maxent	`maxnet`
Linear features	✓	✓	✓
Quadratic features	✓	✓	✓
Product features	✓	✓	✓
Hinge features	✓	✓	✓
Threshold features	✓	✓	✓
Raw/logistic/cloglog output	✓	✓	✓
AUC evaluation	✓	✓	via ENMeval
Permutation importance	✓	✓	via ENMeval
Response curves	✓	✓	partial
MESS maps	✓	✓	–
Clamping / fade-by-clamping	✓	✓	–
Lambda file I/O	✓	✓	–
Streaming raster projection	✓	–	–
Bias grids	✓	✓	via `offset`
Categorical variables	–	✓	✓
Replicate runs / cross-val.	–	✓	via ENMeval
Jackknife variable selection	–	✓	–
Missing data handling	–	✓	✓
SWD-to-raster projection	–	✓	–

Categorical variables, replicate runs, jackknife, and SWD-to-raster projection are not yet implemented. Users requiring these features should continue using Java Maxent or `maxnet`. We plan to add categorical variable support in a future release.

5.6 Development provenance

The translation followed a phased approach across four publicly accessible repositories:

1. `alrobles/Maxent`—a fork of the original `mrmaxent/Maxent` [Phillips et al., 2017] Java source code, where the architecture was analyzed and each Java class was mapped to a C++ target (193 commits).
2. `alrobles/maxentcpp-devel`—the development repository where the C++ port, Rcpp bindings, R interface, tests, and documentation were iteratively built and refined (34 commits).
3. `alrobles/maxentcppCompTest`—the cross-language fidelity test suite containing Java oracle runners, golden trajectory files, and automated comparison scripts (40 commits).
4. `alrobles/maxentcpp`—the release repository with the clean, CRAN-ready package (47 commits).

6 Ecosystem comparison

The R ecosystem contains several packages that provide access to MaxEnt models, each employing a distinct integration strategy:

Table 6: R packages providing MaxEnt access.

Package	MaxEnt integration	Dependency
dismo [Hijmans et al., 2023]	rJava bridge to `maxent.jar`	Java JDK, rJava
kuenm [Cobos et al., 2019]	`system2("java -jar maxent.jar")`	Java JDK
kuenm2 (Cobos et al.)	`glmnet` via forked `maxnet` code	glmnet
wallace [Kass et al., 2018]	Delegates to ENMeval → maxnet or dismo	ENMeval
ENMTools [Warren et al., 2010]	`dismo::maxent()` directly	dismo, rJava
biomod2 [Thuiller et al., 2009]	Java (`system2`) or `maxnet`	Optional Java or maxnet
maxentcpp	Native C++17 via Rcpp	None (self-contained)

These packages can be grouped by their relationship to the MaxEnt algorithm:

- **Black-box wrappers** (dismo, kuenm, ENMTools, biomod2 MAXENT mode): invoke the Java binary and read output files. The optimizer is inaccessible.
- **Approximate reimplementations** (maxnet, kuenm2, biomod2 MAXNET mode): use `glmnet` coordinate descent on the logistic regression formulation. The statistical model is equivalent but the optimization path differs, which can produce numerically different results for threshold and hinge features.
- **Faithful reimplementation** (maxentcpp): ports the original Sequential optimizer to C++ and proves per-iteration numerical equivalence.

The unique contribution of `maxentcpp` is that it is the only package offering a compiled native reimplementation of the actual MaxEnt optimizer—preserving both the statistical model and the optimization algorithm while eliminating the Java dependency.

7 Limitations and inherited assumptions

The Maxent 3.4.4 algorithm that `maxentcpp` faithfully reproduces has well-documented limitations that users should be aware of:

- **Feature explosion.** The number of features (particularly hinge and threshold) grows with the number of environmental variables, which can cause identifiability issues in high-dimensional settings [Elith et al., 2011, Merow et al., 2013].
- **Sensitivity to background sampling.** Maxent's output is influenced by the choice and size of the background sample, which affects the estimated density and can bias predictions in undersampled regions [Phillips and Dudík, 2008].
- ℓ_1 **regularization limitations.** The sequential coordinate ascent with ℓ_1 penalties produces sparse models but does not guarantee selection of the "correct" features under high correlation among predictors [Hastie et al., 2015].
- **No principled uncertainty quantification.** Unlike Bayesian approaches or bootstrap-based methods, the Maxent optimizer produces point estimates without confidence intervals.

A principled alternative would be to reformulate the Maxent objective using modern convex optimization tooling (e.g., proximal gradient methods, ADMM) or Bayesian inference. However, such reformulations would produce different results than the widely used Java implementation, breaking comparability with published analyses. The design choice in `maxentcpp` is explicitly to preserve this comparability.

8 Research impact statement

`maxentcpp` is designed as a numerically faithful alternative to Java Maxent for R-based ecological workflows. The term "drop-in replacement" applies specifically to the implemented features listed in the completeness table above: for those features, `maxentcpp` produces results identical to Java Maxent (to within IEEE 754 double-precision limits) given the same inputs, defaults, and random seed. Features not yet implemented (categorical variables, replicate runs, jackknife) require continued use of Java Maxent.

The package is distributed under the MIT license (compatible with Eigen's MPL2 and RcppEigen's GPL-2 via the "or later" clause), with full source code, a pkgdown documentation website (`https://alrobles.github.io/maxentcpp/`), and four vignettes covering a getting-started walkthrough, the mathematical foundations of Maxent features, a comparative motivation document, and a virtual species validation study. Continuous integration via GitHub Actions runs `R CMD check` on Ubuntu, macOS, and Windows across R release and R-devel. The test suite comprises over 20 test files (∼4 800 lines) covering feature generation, optimization convergence, spatial projection, Java numerical equivalency, model diagnostics, and end-to-end workflows.

By providing a dependency-free, memory-efficient, and numerically faithful Maxent implementation, `maxentcpp` lowers the barrier for large-scale biodiversity assessments, climate-change projections, and conservation prioritization studies that rely on presence-only species distribution models.

9 AI usage disclosure

Generative AI tools were used during the development of `maxentcpp` and the preparation of this manuscript, as detailed below.

Tools used.

- **GitHub Copilot** (GPT-4-based, 2025 version): code scaffolding and boilerplate generation during the initial porting phases in `alrobles/Maxent` and `alrobles/maxentcpp-devel`.
- **Devin** (Cognition AI, 2025–2026): incremental feature implementation, test writing, documentation generation, CRAN compliance fixes, CI configuration, and manuscript drafting.
- **Claude** (Anthropic, 2025–2026): complex refactoring, cross-cutting documentation, and code review.

Nature and scope of assistance. AI tools contributed to: C++ code generation and refactoring from Java source, Rcpp binding scaffolding, R wrapper functions, `testthat` test scaffolding and expansion, `roxygen2` documentation, vignette drafting, pkgdown site configuration, CI workflow setup, CRAN compliance review, and manuscript drafting. The core algorithmic C++ classes (`Sequential`, `FeaturedSpace`, feature types) were translated from Java with AI assistance but under direct human supervision: each class was mapped line-by-line from the corresponding Java source, reviewed for correctness against the Java reference, and validated through the `maxentcppCompTest` trajectory comparison framework. Approximately 60% of the C++ algorithmic lines were initially drafted by AI tools and subsequently reviewed and corrected by the human author; the remaining 40% were written directly by the author, particularly the performance-critical optimizer inner loops and numerical edge cases.

Maintainability. The human author (Á. L. Robles Fernández) can independently explain and modify every algorithmically significant class. As evidence: the `Sequential::run()` optimizer, `good_alpha()`, `delta_loss_bound()`, `newton_step_feature()`, and `do_parallel_update()` methods are documented with line-by-line references to the corresponding Java source (e.g., "mirrors Sequential.java:294..342"), and the author has independently debugged numerical discrepancies during the fidelity testing process.

Confirmation of review. All AI-generated code, tests, documentation, and manuscript text were reviewed, edited, and validated by the human author (Á. L. Robles Fernández), who made all core design decisions including the algorithm translation strategy, API design, numerical fidelity requirements, and the phased development architecture. The full development history is publicly available in the four repositories listed above.

Acknowledgements

The author thanks Steven J. Phillips, Robert P. Anderson, and Robert E. Schapire for creating the original Maxent software and making the source code publicly available under an MIT license. The `Rcpp`, `RcppEigen`, and `terra` package maintainers are gratefully acknowledged for providing the infrastructure that makes `maxentcpp` possible.

References

Douglas Bates and Dirk Eddelbuettel. Fast and elegant numerical linear algebra using the RcppEigen package. *Journal of Statistical Software*, 52(5):1–24, 2013. doi: 10.18637/jss.v052.i05.

Marlon E. Cobos, A. Townsend Peterson, Narayani Barve, and Luis Osorio-Olvera. kuenm: an R package for detailed development of ecological niche models using Maxent. *PeerJ*, 7:e6281, 2019. doi: 10.7717/peerj.6281.

Dirk Eddelbuettel. *Seamless R and C++ Integration with Rcpp*. Springer, New York, 2013. doi: 10.1007/978-1-4614-6868-4.

Dirk Eddelbuettel and Romain François. Rcpp: Seamless R and C++ integration. *Journal of Statistical Software*, 40(8):1–18, 2011. doi: 10.18637/jss.v040.i08.

Jane Elith, Steven J. Phillips, Trevor Hastie, Miroslav Dudík, Yung En Chee, and Colin J. Yates. A statistical explanation of MaxEnt for ecologists. *Diversity and Distributions*, 17(1):43–57, 2011. doi: 10.1111/j.1472-4642.2010.00725.x.

Ángel Luis Robles Fernández. maxentcppCompTest: Cross-language fidelity tests for maxentcpp, 2025. URL https://github.com/alrobles/maxentcppCompTest.

Gaël Guennebaud, Benoît Jacob, et al. Eigen v3, 2010. URL https://eigen.tuxfamily.org.

Kimberly R. Hall, Ranjan Anantharaman, Vincent A. Landau, Melissa Clark, Brett G. Dickson, Aaron Jones, Jim Platt, Alan Edelman, and Viral B. Shah. Circuitscape in Julia: Empowering dynamic approaches to connectivity assessment. *Land*, 10(3):301, 2021. doi: 10.3390/land10030301.

Trevor Hastie, Robert Tibshirani, and Martin Wainwright. *Statistical Learning with Sparsity: The Lasso and Generalizations*. CRC Press, 2015. doi: 10.1201/b18401.

Robert J. Hijmans. *terra: Spatial Data Analysis*, 2024. URL https://CRAN.R-project.org/package=terra. R package version 1.7-78.

Robert J. Hijmans, Steven Phillips, John Leathwick, and Jane Elith. *dismo: Species Distribution Modeling*, 2023. URL https://CRAN.R-project.org/package=dismo. R package version 1.3-14.

Jamie M. Kass, Bruno Vilela, Matthew E. Aiello-Lammens, Robert Muscarella, Cory Merow, and Robert P. Anderson. wallace: A flexible platform for reproducible modeling of species niches and distributions built for community expansion. *Methods in Ecology and Evolution*, 9 (4):1151–1156, 2018. doi: 10.1111/2041-210X.12945.

Jamie M. Kass, Robert Muscarella, Peter J. Galante, Corentin L. Bohl, Gonzalo E. Pinilla-Buitrago, Robert A. Boria, Mariano Soley-Guardia, and Robert P. Anderson. ENMeval 2.0: Redesigned for customizable and reproducible modeling of species' niches and distributions. *Methods in Ecology and Evolution*, 12(9):1602–1608, 2021. doi: 10.1111/2041-210X.13628.

Boris Leroy, Robin Delsol, Bernard Hugueny, Christine N. Meynard, Céline Baroni, et al. virtualspecies, an R package to generate virtual species distributions. *Ecography*, 39(6): 599–607, 2016. doi: 10.1111/ecog.01388.

Cory Merow, Matthew J. Smith, and John A. Silander, Jr. A practical guide to MaxEnt for modeling species' distributions: what it does, and why inputs and settings matter. *Ecography*, 36(10):1058–1069, 2013. doi: 10.1111/j.1600-0587.2013.07872.x.

Emmanuel Nyenah, Petra Döll, Martina Flörke, Leon Mühlenbruch, Lasse Nissen, and Robert Reinecke. The process and value of reprogramming a legacy global hydrological model. *Geoscientific Model Development*, 18:5635–5653, 2025. doi: 10.5194/gmd-18-5635-2025.

Steven J. Phillips. *maxnet: Fitting 'Maxent' Species Distribution Models with 'glmnet'*, 2024. URL https://CRAN.R-project.org/package=maxnet. R package version 0.1.4.

Steven J. Phillips and Miroslav Dudík. Modeling of species distributions with Maxent: new extensions and a comprehensive evaluation. *Ecography*, 31(2):161–175, 2008. doi: 10.1111/j.0906-7590.2008.5203.x.

Steven J. Phillips, Miroslav Dudík, and Robert E. Schapire. A maximum entropy approach to species distribution modeling. In *Proceedings of the Twenty-First International Conference on Machine Learning*, ICML '04, pages 655–662, New York, NY, USA, 2004. ACM. doi: 10.1145/1015330.1015412.

Steven J. Phillips, Robert P. Anderson, and Robert E. Schapire. Maximum entropy modeling of species geographic distributions. *Ecological Modelling*, 190(3–4):231–259, 2006. doi: 10.1016/j.ecolmodel.2005.03.026.

Steven J. Phillips, Robert P. Anderson, Miroslav Dudík, Robert E. Schapire, and Mary E. Blair. Opening the black box: an open-source release of Maxent. *Ecography*, 40(7):887–893, 2017. doi: 10.1111/ecog.03049.

Robert M. Scheller, Brian R. Sturtevant, Eric J. Gustafson, Brendan C. Ward, and David J. Mladenoff. Increasing the reliability of ecological models using modern software engineering techniques. *Frontiers in Ecology and the Environment*, 8(5):253–260, 2009. doi: 10.1890/080141.

Wilfried Thuiller, Damien Georges, and Robin Engler. biomod2: Ensemble platform for species distribution modeling. *R package*, 2009. URL https://CRAN.R-project.org/package=biomod2.

Dan L. Warren, Russell E. Glor, and Michael Turelli. ENMTools: a toolbox for comparative studies of environmental niche models. *Ecography*, 33(3):607–611, 2010. doi: 10.1111/j.1600-0587.2009.06142.x.